
Los Sabrossos

Contents

1	Math	1
1.1	Sieve of Eratosthenes	1
1.2	Binary Exponentiation	1
1.3	Extended Euclides	1
1.4	Fibonacci Log(n)	1
1.5	Modular Inverse	2
1.6	Pollard Rho Rabin Miller	2
2	Geometry	3
2.1	Convex Hull	3
2.2	Point	3
2.3	Closest Pair of Points Lineal	4
2.4	Closest Pair of Points Nlogn	5
3	Strings	6
3.1	KMP	6
3.2	Z Function	6
3.3	Trie	7
3.4	Suffix Array	7
3.5	Aho Corasick	9
3.6	Hashing	11
3.7	Manacher	11
4	Bit Manipulation	11
4.1	Binary Representation	11
4.2	Bit Mask	12
4.3	Bits Prendidos En Un Numero X En Una Posicion Pos	12
5	Data Structures	12
5.1	Segment Tree	12
5.2	Segment Tree Lazy Propagation	13
5.3	BIT Fenwick Tree	14
5.4	Bit 2D	14
5.5	Disjoint Set Union	15
5.6	Implicit Treap	16
5.7	Ordered Set	17
5.8	Sparse Table	17
5.9	Treap	17
6	Dynamic Programing	18
6.1	Bounded Knapsack	18
6.2	SOS DP	19
6.3	Fastfibo	19
7	Flow	20
7.1	Dinic	20
8	Graph Algorithms	21
8.1	Dijkstra	21
8.2	Kruskal MST	21
8.3	Topological Sort Kahn Algorithm	22

9	Tree Algorithms	22
9.1	Euler Tour	22
9.2	Binary Lifting	23
9.3	Centroid Decomposition	23
9.4	Lowest Common Ancestor	25
10	Utilities	25
10.1	Int128	25
10.2	Merge Sort	26
10.3	Prefix Sum 2D	26
10.4	Random	27
10.5	Plantilla Python	27
10.6	Pragma	27

1. Math

1.1. Sieve of Eratosthenes

```
1  const int N = 1e7;
2  int sieve[N];
3  void make(){
4      for(int i = 0; i < N; i++) sieve[i] = 1;
5      sieve[0] = sieve[1] = 0;
6      for (int i = 2; i < N; i++)
7          {
8              if(sieve[i]){
9                  for (int j = i*i; j < N; j+=i)
10                     {
11                         sieve[j] = 0;
12                     }
13             }
14         }
15 }
```

1.2. Binary Exponentiation

```
1  long long pote(long long a, long long b, long long mod){
2      long long ans = 1ll;
3      while(b){
4          if(b&1) ans = (ans * a)%mod;
5          a = (a * a)%mod;
6          b>>=1;
7      }
8      return ans;
9  }
10 // a-1 = amod-2 = pote(a, mod-2);
```

1.3. Extended Euclides

```
1  //Encuentra
2  //valores para x e y
3  //A*x + B*y = gcd(A, B)
4  ll extgcd(ll a, ll b, ll &x, ll &y){
5      if(b == 0){
6          x = 1;
7          y = 0;
8          return a;
9      }
10
11      ll x1, y1;
12      ll g = extgcd(b, a % b, x1, y1);
13      x = y1;
14      y = x1 - (a / b) * y1;
15      return g;
16  }
17
18  void solve(){
19      ll A, B, C;
20      cin >> A >> B >> C;
21      ll x, y;
22      ll g = extgcd(A, B, x, y);
23      if(C % g != 0){
24          cout << "-1\n";
25          return;
26      }
27      ll k = -C / g;
28      x *= k;
29      y *= k;
30      cout << x << " " << y << '\n';
31 }
```

1.4. Fibonacci Log(n)

```
1  long long int fibo(int n) {
2      long long int a, b, x, y, tmp;
3      a = x = tmp = 0;
4      b = y = 1;
5      while (n != 0)
6          if (n % 2 == 0) {
```

```

7         tmp = a * a + b * b;
8         b = b * a + a * b + b * b;
9         a = tmp;
10        n = n / 2;
11    } else {
12        tmp = a * x + b * y;
13        y = b * x + a * y + b * y;
14        x = tmp;
15        n = n - 1;
16    }
17    // x = fibo(n-1)
18    return y; // y = fibo(n)
19 }

```

1.5. Modular Inverse

```

1  const long long N = 1e6 + 10;
2  const int MOD = 1e9+7;
3  long long fact[N];
4  long long invfact[N];
5  void init(){
6      fact[0] = 1;
7      for(long long i = 1; i < N; i++){
8          fact[i] = fact[i-1] * i % MOD;
9      }
10     invfact[N-1] = pote(fact[N-1], MOD-2);
11
12     for(long long i = N-2; i >= 0; i--){
13         invfact[i] = invfact[i+1] * (i+1) % MOD;
14     }
15
16     long long nCk(long long n, long long k){
17         if(k < 0 || k > n) return 0;
18         return fact[n] * invfact[k] % MOD * invfact[n-k] % MOD;
19     }

```

1.6. Pollard Rho Rabin Miller

```

1  map<long long, long long> fact;
2
3  long long mcd(long long a, long long b) {
4      a = abs(a);
5      b = abs(b);
6      while (a != 0) {
7          long long r = b % a;
8          b = a;
9          a = r;
10     }
11     return b;
12 }
13 //Rabin Miller
14 bool primo(long long n) {
15     if (n < 2) return false;
16     if (n == 2) return true;
17     long long D = n - 1, S = 0;
18     while (D % 2 == 0) {
19         D /= 2;
20         S++;
21     }
22     static const long long STEPS = 1611;
23     for(long long pasos = 0; pasos < STEPS; pasos++){
24         const long long A = 1 + rand() % (n - 1);
25         long long M = pote(A, D, n);
26         if (M == 1 || M == (n - 1)) goto next;
27         for(long long k = 0; k < S-1; k++){
28             M = ((M) * M) % n;
29             if (M == (n - 1)) goto next;
30         }
31         return false;
32     next:;
33     }
34     return true;
35 }
36
37 //Rho de pollard
38 long long factor(long long n) {
39     static long long A, B;
40     A = 1 + rand() % (n - 1);
41     B = 1 + rand() % (n - 1);
42     #define fun(x) (((x) * (x + B)) % n + A)
43

```

```
44     long long x = 2, y = 2, d = 1;
45     while (d == 1 || d == -1) {
46         x = fun(x);
47         y = fun(fun(y));
48         d = mcd(x - y, n);
49     }
50     return abs(d);
51 }
52
53 void factorize(long long n){
54     assert(n > 0);
55     while (n > 1 && !primo(n)) {
56         long long fx;
57         do {
58             fx = factor(n);
59         } while (fx == n);
60
61         n /= fx;
62         factorize(fx);
63
64         for (auto &it : fact) {
65             while (n % it.first == 0) {
66                 n /= it.first;
67                 it.second++;
68             }
69         }
70     }
71     if (n > 1) fact[n]++;
72 }
```

2. Geometry

2.1. Convex Hull

```
1 // Devuelve sentido horario
2 vector<point> hull(vector<point> p)
3 {
4     int n = p.size();
5     vector<point> h;
6     sort(p.begin(), p.end());
7     for(int i = 0; i < n; i++){
8         while(h.size() >= 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
9             h.pop_back();
10        }
11        h.push_back(p[i]);
12    }
13    h.pop_back();
14    int k = h.size();
15    for(int i = n-1; i >= 0; i--){
16        while(h.size() >= k + 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
17            h.pop_back();
18        }
19        h.pb(p[i]);
20    }
21    h.pop_back();
22    return h;
23 }
24 }
```

2.2. Point

```
1 struct point
2 {
3     int x, y;
4     point() {}
5     point(int x, int y): x(x), y(y) {}
6     point operator -(point p) {return point(x - p.x, y - p.y);}
7     point operator +(point p) {return point(x + p.x, y + p.y);}
8     int sq() const{return x * x + y * y;}
9     double abs() const {return sqrt(sq());}
10    int operator ^(point p) {return x * p.y - y * p.x;}
11    int operator *(point p) {return x * p.x + y * p.y;}
12    point operator *(int a) {return point(x * a, y * a);}
13    bool operator <(const point& p) const {return x == p.x ? y < p.y : x < p.x;}
14    int left(point a, point b) {
15        return ((b - a) ^ (*this - a)); // 0, -, +
16    }
17 };
```

```

18 ostream& operator<<(ostream& os, const point& p) {
19     return os << "(" << p.x << ", " << p.y << ")\n";
20 }
21
22
23 void polarSort(vector<point>& v) {
24     sort(v.begin(), v.end(), [] (point a, point b) {
25         const point origin{0, 0};
26         bool ba = a < origin, bb = b < origin;
27         if (ba != bb) { return ba < bb; }
28         return (a^b) > 0;
29     });
30 }

```

2.3. Closest Pair of Points Lineal

```

1 struct pt {
2     ll x, y;
3     bool operator == (const pt & o) const {
4         return x == o.x and y == o.y;
5     }
6 };
7
8 struct CustomHashPoint {
9     size_t operator()(const pt & p) const {
10         static
11         const uint64_t C =
12             chrono::steady_clock::now().time_since_epoch().count();
13         uint64_t x = p.x;
14         uint64_t y = p.y;
15         x ^= x >> 30;
16         x *= 0xbf58476d1ce4e5b9;
17         x ^= x >> 27;
18         x *= 0x94d049bb133111eb;
19         x ^= x >> 31;
20         y ^= y >> 30;
21         y *= 0xbf58476d1ce4e5b9;
22         y ^= y >> 27;
23         y *= 0x94d049bb133111eb;
24         y ^= y >> 31;
25         return C ^ (x + (y << 1));
26     }
27 };
28
29 ll dist2(pt a, pt b) {
30     ll x = a.x - b.x;
31     ll y = a.y - b.y;
32     return x * x + y * y;
33 }
34
35 pair < int, int > closest_pair(vector < pt > p) {
36     int n = p.size();
37     unordered_map < pt, int, CustomHashPoint > mp;
38     // puntos repetidos
39     for (int i = 0; i < n; i++) {
40         if (mp.count(p[i]))
41             return {
42                 mp[p[i]],
43                 i
44             };
45         mp[p[i]] = i;
46     }
47     mt19937 rng(
48         chrono::steady_clock::now().time_since_epoch().count()
49     );
50     auto rnd = [&]() {
51         return uniform_int_distribution < int > (0, n - 1)(rng);
52     };
53     ll d2 = dist2(p[0], p[1]);
54     pair < int, int > ans = {
55         0,
56         1
57     };
58     auto upd = [&](int i, int j) {
59         ll nd = dist2(p[i], p[j]);
60         if (nd < d2) {
61             d2 = nd;
62             ans = {
63                 i,
64                 j
65             };
66         }
67     };
68     // conseguir una distancia inicial razonable

```

```

69     for (int it = 0; it < 20; it++) {
70         int i = rnd();
71         int j = rnd();
72         while (i == j) j = rnd();
73         upd(i, j);
74     }
75     ll d = sqrtl((long double) d2) + 1;
76     unordered_map < pt, vector < int > , CustomHashPoint > grid;
77     grid.reserve(2 * n);
78     for (int i = 0; i < n; i++) {
79         grid[{
80             p[i].x / d,
81             p[i].y / d
82         }].push_back(i);
83     }
84     for (auto & [c, v]: grid) {
85         // misma celda
86         for (int i = 0; i < (int) v.size(); i++) {
87             for (int j = i + 1; j < (int) v.size(); j++) {
88                 upd(v[i], v[j]);
89             }
90         }
91         // celdas vecinas
92         for (int dx = -1; dx <= 1; dx++) {
93             for (int dy = -1; dy <= 1; dy++) {
94                 if (dx == 0 and dy == 0) continue;
95                 pt to = {
96                     c.x + dx,
97                     c.y + dy
98                 };
99                 auto it = grid.find(to);
100                 if (it == grid.end()) continue;
101                 for (int i: v) {
102                     for (int j: it->second) {
103                         upd(i, j);
104                     }
105                 }
106             }
107         }
108     }
109     return ans;
110 }

```

2.4. Closest Pair of Points Nlogn

```

1  struct pt {
2      ll x, y;
3      int id;
4  };
5
6  ll dist2(pt a, pt b) {
7      ll x = a.x - b.x;
8      ll y = a.y - b.y;
9      return x * x + y * y;
10 }
11
12 pair < int, int > closest_pair(vector < pt > p) {
13     int n = p.size();
14     sort(p.begin(), p.end(), [](pt a, pt b) {
15         if (a.x != b.x) return a.x < b.x;
16         return a.y < b.y;
17     });
18     set < pair < ll, int >> s; // {y, id}
19     ll d2 = dist2(p[0], p[1]);
20     pair < int, int > ans = {
21         p[0].id,
22         p[1].id
23     };
24     int l = 0;
25     for (int i = 0; i < n; i++) {
26         ll d = sqrtl((long double) d2) + 1;
27         // sacar puntos que ya est n demasiado lejos en x
28         while (l < i and p[i].x - p[l].x >= d) {
29             s.erase({
30                 p[l].y,
31                 p[l].id
32             });
33             l++;
34         }
35         // buscar puntos con y cerca
36         auto it = s.lower_bound({
37             p[i].y - d,
38             -1
39         });

```

```

40     while (it != s.end() and it -> first <= p[i].y + d) {
41         int j = it -> second;
42         ll nd = dist2(p[i], p[j]);
43         if (nd < d2) {
44             d2 = nd;
45             ans = {
46                 p[i].id,
47                 p[j].id
48             };
49             d = sqrtl((long double) d2) + 1;
50         }
51         it++;
52     }
53     s.insert({
54         p[i].y,
55         p[i].id
56     });
57 }
58 return ans;
59 }

```

3. Strings

3.1. KMP

```

1  vector<int> kmp(string s){
2      int n = s.size();
3      vector<int> pi(n);
4      for(int i = 1; i < n; i++){
5          int j = pi[i-1];
6          while(j > 0 and s[i] != s[j]){
7              j = pi[j-1];
8          }
9          if(s[i] == s[j]){
10             j++;
11         }
12         pi[i] = j;
13     }
14     return pi;
15 }
16 //Pattern Matching
17 vector<int> ocorrencia(string texto, string patron) {
18     vector<int> P = kmp(patron);
19     int k = 0;
20     vector<int> M;
21     repl(i,0,texto.size()) {
22         while (k > 0 and patron[k] != texto[i]) k = P[k - 1];
23         if (patron[k] == texto[i]) k++;
24         if (k == patron.size()) {
25             M.pb(i - k + 1);
26             k = P[k - 1];
27         }
28     }
29     return M;
30 }
31
32 //Automata
33 void genAut(string &s) {
34     int n = s.size();
35
36     vector<vector<int>>> aut(n+1, vector<int>(26));
37     vector<int> pi = kmp(s);
38
39     for(int i = 0 ; i < n; i++){
40         aut[i][s[i]-'a'] = i+1;
41     }
42
43     for(int i = 0; i < n+1; i++){
44         for(int c = 0; c < 26; c++){
45             if (aut[i][c]) continue;
46             if (i < n and s[i] == 'a' + c) aut[i][c] = i+1;
47             else if (i > 0) aut[i][c] = aut[pi[i-1]][c];
48             else aut[i][c] = 0;
49         }
50     }
51 }

```

3.2. Z Function

```

1  vector<int> z_function(string s){
2      int n = s.size();
3      vector<int> z(n,0);
4      int l = 0;
5      int r = 0;
6      repl(i,1,n){
7          if(i < r){
8              z[i] = min(r - i, z[i - l]);
9          }
10         while(i + z[i] < n and s[z[i]] == s[i + z[i]]){
11             z[i]++;
12         }
13         if(i + z[i] > r){
14             l = i;
15             r = i + z[i];
16         }
17     }
18     return z;
19 }

```

3.3. Trie

```

1  const int E = 26;
2
3  struct AC {
4      int nodes;
5      vector<array<int, E>>go;
6      vector<bool>terminal;
7
8      AC(){
9          add_node();
10         nodes = 1;
11     }
12
13     void add_node(){
14         terminal.emplace_back();
15         go.emplace_back();
16     }
17
18     void insert(string &s){
19         int pos = 0;
20         for(char ch : s){
21             int c = ch - '0';
22             if(go[pos][c] == 0){
23                 go[pos][c] = nodes++;
24                 add_node();
25             }
26             pos = go[pos][c];
27         }
28         terminal[pos] = true;
29     }
30
31 };

```

3.4. Suffix Array

```

1  vector<int> suffix_array(string s) {
2      s += char(0);
3      const int n = s.size();
4      vector<int> a(n);
5      iota(a.begin(), a.end(), 0);
6      sort(a.begin(), a.end(), [&] (int i, int j){
7          return s[i] < s[j];
8      });
9      vector<int> mapping(n);
10     mapping[a[0]] = 0;
11     for(int i = 1; i < n; i++){
12         mapping[a[i]] = mapping[a[i-1]] + (s[a[i-1]] != s[a[i]]);
13     }
14     int len = 1;
15     vector<int> sbs(n);
16     vector<int> head(n);
17     vector<int> new_mapping(n);
18     while(len < n){
19         for(int i = 0; i < n; i++) sbs[i] = (a[i] + n - len) % n;
20         for(int i = n-1; i >= 0; i--) head[mapping[a[i]]] = i;
21         for(int i = 0; i < n; i++){
22             int x = sbs[i];
23             a[head[mapping[x]]++] = x;
24         }

```

```

25     new_mapping[a[0]] = 0;
26     for(int i = 1; i < n; i++){
27         if(mapping[a[i-1]] != mapping[a[i]]){
28             new_mapping[a[i]] = new_mapping[a[i-1]]+1;
29         }
30         else{
31             int pre = mapping[(a[i-1] + len)%n];
32             int cur = mapping[(a[i]+len) % n];
33             new_mapping[a[i]] = new_mapping[a[i-1]] + (pre!=cur);
34         }
35     }
36     len <= 1;
37     swap(mapping, new_mapping);
38 }
39 return vector<int>(a.begin()+1, a.end()); //ignorar a[0] que es el centinela
40 }
41
42 vector<int> lcp_array(vector<int>&a, string s){
43     const int n = s.size();
44     vector<int> rank(n);
45     for(int i = 0; i < n; i++) rank[a[i]] = i;
46     int k = 0;
47     vector<int> lcp(n);
48     for(int i = 0; i < n; i++){
49         if(rank[i] == n-1){
50             k = 0;
51             continue;
52         }
53         int j = a[rank[i]+1];
54         while(i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
55         lcp[rank[i]] = k;
56         if(k) k--;
57     }
58     lcp.pop_back();
59     return lcp;
60 }
61
62 void solve(){
63     //Numero de distintos substr en un string  $n*(n+1)/2$  - sumatoria del LCP
64     //verificar si un string es substring de otro con BS en SA
65     //encontrar el longest common substring en 2 strings(ESTA IMPLEMENTACION)
66     string s, t;
67     cin >> s >> t;
68     if(s.size() > t.size()){
69         swap(s, t);
70     }
71     int n = s.size();
72     int m = t.size();
73     string st = s+'#'+t;
74     vector<int> sa = suffix_array(st);
75     vector<int> lcp = lcp_array(sa, st);
76     // print(sa);
77     // print(lcp);
78     vector<int> tr(n+m);
79     repl(i,0,n+m){
80         if(sa[i] >= n and sa[i+1] >= n){
81             tr[i] = 0;
82             continue;
83         }
84         if(sa[i] < n and sa[i+1] < n){
85             tr[i] = 0;
86             continue;
87         }
88         int mn = min(sa[i], sa[i+1]);
89         int tam1 = mn + lcp[i];
90         // dbg(tam1);
91         tr[i] = lcp[i];
92         if(tam1 > n){
93             dbg(tam1);
94             tr[i] = lcp[i] - (tam1-n);
95         }
96     }
97     // print(tr);
98     int mx = *max_element(tr.begin(), tr.end());
99     repl(i,0,n+m){
100         // dbg(tr[i]);
101         if(tr[i] == mx){
102             string ans = st.substr(sa[i], mx);
103             cout << ans << '\n';
104             return;
105         }
106     }
107 }
108 }
109 //Sabrossus

```

3.5. Aho Corasick

```

1 //AhoCorasick para saber si un string s existe en un texto T
2 const int E = 26;
3 const int N = 1e6+10;
4 int cur;
5 int nodoTerminal[N]; //se guarda el nodo terminal del string i
6 set<int>st; //se guarda que nodos son terminales que aparecen en el texto T
7
8 struct AC {
9     int nodes;
10    vector<int> suf, super;
11    vector<array<int, E>> go;
12    vector<bool> terminal;
13    AC(){
14        add_node();
15        nodes = 1;
16    }
17    void add_node(){
18        terminal.emplace_back();
19        go.emplace_back();
20        super.emplace_back();
21        suf.emplace_back();
22    }
23
24    void insert(string &s){
25        int pos = 0;
26        for(char ch : s){
27            int c = ch - 'a';
28            if(go[pos][c] == 0){
29                go[pos][c] = nodes++;
30                add_node();
31            }
32            pos = go[pos][c];
33        }
34        terminal[pos] = true;
35        nodoTerminal[cur] = pos;
36    }
37
38    void build(){
39        queue<int> Q;
40        Q.emplace(0);
41        while(!Q.empty()){
42            int u = Q.front();
43            Q.pop();
44            super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] : super[suf[u]]);
45            for(int c = 0; c < E; c++){
46                if(go[u][c]){
47                    int v = go[u][c];
48                    Q.emplace(v);
49                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
50                }
51                else{
52                    go[u][c] = u == 0 ? 0 : go[suf[u]][c];
53                }
54            }
55        }
56    }
57 };
58
59 void mark(int u, AC &ac){
60     if(u == 0) return;
61     mark(ac.super[u], ac);
62     if(ac.terminal[u]){
63         st.insert(u);
64         ac.terminal[u] = false;
65     }
66     ac.super[u] = 0;
67 }
68
69 void solve(){
70     int n;
71     cin >> n;
72     AC ac;
73     repl(i,0,n){
74         string s;
75         cin >> s;
76         cur = i;
77         ac.insert(s);
78     }
79     string t; //texto grande
80     cin >> t;
81     ac.build();
82     int p = 0;
83     for(auto x : t){
84         int c = x - 'a';

```

```

85     p = ac.go[p][c];
86     mark(p, ac);
87 }
88 repl(i,0,n){
89     if(st.count(nodoTerminal[i])){
90         cout << "YES\n";
91     }
92     else{
93         cout << "NO\n";
94     }
95 }
96 }
97
98 //-----
99
100 //AhoCorasick para Frecuencias
101 const int E = 26;
102 const int N = 1e6+10;
103 int nodoTerminal[N];
104 int cur;
105 struct AC {
106     int nodes;
107     vector<int> suf, freq, by_level;
108     vector<bool> terminal;
109     vector<array<int,E>> go;
110
111     AC(){
112         add_node();
113         nodes = 1;
114     }
115     void add_node(){
116         terminal.emplace_back();
117         suf.emplace_back();
118         go.emplace_back();
119         freq.emplace_back();
120     }
121
122     void insert(string &s){
123         int pos = 0;
124         for(char ch:s){
125             int c = ch - 'a';
126             if(go[pos][c] == 0){
127                 go[pos][c] = nodes++;
128                 add_node();
129             }
130             pos = go[pos][c];
131         }
132         terminal[pos] = true;
133         nodoTerminal[cur] = pos;
134     }
135
136     void build(){
137         queue<int> Q;
138         Q.emplace(0);
139         while(!Q.empty()){
140             int u = Q.front();
141             Q.pop();
142             by_level.emplace_back(u);
143             for(int c = 0; c < E; c++){
144                 if(go[u][c]){
145                     int v = go[u][c];
146                     Q.emplace(v);
147                     suf[v] = u == 0 ? 0 : go[suf[u]][c];
148                 }
149                 else{
150                     go[u][c] = u == 0 ? 0 : go[suf[u]][c];
151                 }
152             }
153         }
154     }
155 };
156
157 void solve(){
158     int n;
159     cin >> n;
160     AC ac;
161     repl(i,0,n){
162         string s;
163         cin >> s;
164         cur = i;
165         ac.insert(s);
166     }
167     string t;
168     cin >> t;
169     ac.build();
170     int p = 0;

```

```

171     for(char ch:t){
172         int c = ch-'a';
173         p = ac.go[p][c];
174         ac.freq[p]++;
175     }
176     for(int i = (int)ac.by_level.size()-1; i > 0; i--){
177         int x = ac.by_level[i];
178         ac.freq[ac.suf[x]] += ac.freq[x];
179     }
180     repl(i,0,n){
181         int termi = nodoTerminal[i];
182         cout << ac.freq[termi] << '\n';
183     }
184 }

```

3.6. Hashing

```

1  struct Hash {
2      long long P=1777771,MOD[2],PI[2];
3      vector<long long> h[2],pi[2];
4      Hash(string& s){
5          MOD[0]=999727999;
6          MOD[1]=107077777;
7          PI[0]=325255434;
8          PI[1]=10018302;
9          for(int k = 0; k < 2; k++) {
10             h[k].resize(s.size()+1);
11             pi[k].resize(s.size()+1);
12         }
13         for(int k = 0; k < 2; k++){
14             h[k][0] = 0;
15             pi[k][0] = 1;
16             long long p = 1;
17             for(int i = 1; i < s.size()+1; i++){
18                 h[k][i] = (h[k][i-1]+p*s[i-1])%MOD[k];
19                 pi[k][i] = (1LL*pi[k][i-1]*PI[k])%MOD[k];
20                 p = (p*P)%MOD[k];
21             }
22         }
23     }
24     long long get(int s, int e){
25         long long h0 = (h[0][e]-h[0][s]+MOD[0])%MOD[0];
26         h0 = (1LL*h0*pi[0][s])%MOD[0];
27         long long h1 = (h[1][e]-h[1][s]+MOD[1])%MOD[1];
28         h1 = (1LL*h1*pi[1][s])%MOD[1];
29         return (h0<<32)|h1;
30     }
31 };

```

3.7. Manacher

```

1  const int N = 1e5 + 10;
2  int odd[N]; //odd[i] = max odd palindrome centered on i
3  int even[N]; //even[i] = max even palindrome centered on i
4  void manacher(string & s) {
5      int l = 0, r = -1, n = s.size();
6      for (int i = 0; i < n; i++) {
7          int k = i > r ? 1 : min(odd[l + r - i], r - i);
8          while (i + k < n and i - k >= 0 and s[i + k] == s[i - k]) k++;
9          odd[i] = k--;
10         if (i + k > r) l = i - k, r = i + k;
11     }
12     l = 0;
13     r = -1;
14     for (int i = 0; i < n; i++) {
15         int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
16         k++;
17         while (i + k <= n and i - k >= 0 and s[i + k - 1] == s[i - k]) k++;
18         even[i] = --k;
19         if (i + k - 1 > r) l = i - k, r = i + k - 1;
20     }
21 }

```

4. Bit Manipulation

4.1. Binary Representation

```
1 string tobit(long long x) {
2     string a(64, '0');
3     for (int i = 63; i >= 0; --i) {
4         if (x & 1) a[i] = '1';
5         x >>= 1;
6     }
7     return a;
8 }
```

4.2. Bit Mask

```
1 for(int mask = 0; mask < (1 << n); mask++) { //(1 << n) = 0(2^n)
2     for(int i = 0; i < n; i++) { // 0(n)
3         //verificamos si el i-esimo bit esta encendido
4         if(mask & (1 << i)) {
5             cout << v[i] << " ";
6         }
7     }
8 }
```

4.3. Bits Prendidos En Un Numero X En Una Posicion Pos

```
1 //retorna el numero de bits prendidos
2 //en una posicion pos desde 0 hasta el numero num
3 //tambien se puede usar en rangos asi:
4 //prendidos(k, l) - prendidos(L-1, l);
5 ll prendidos(ll num, ll pos){
6     if(num <= 0){
7         return 0;
8     }
9     ll pote = (1ll << pos);
10    ll blocks = (num + 1) / pote;
11    ll ones = blocks / 2;
12    ones *= pote;
13    if(blocks & 1){
14        ones += (num + 1) % pote;
15    }
16    return ones;
17 }
18 //Sabrosson
```

5. Data Structures

5.1. Segment Tree

```
1 const int N = 1e5;
2
3 struct info {
4     int num;
5 };
6
7 info ST[4 * N];
8
9 info ope(info u, info v) {
10     return {u.num + v.num};
11 }
12
13 void build(const vector<int> &v, int in = 1, int L = 0, int R = n - 1) {
14     if (L == R) {
15         ST[in] = {v[L]}; // Asignamos el valor desde el vector
16     } else {
17         int mid = (L + R) >> 1;
18         build(v, 2 * in, L, mid);
19         build(v, (2 * in) + 1, mid + 1, R);
20         ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
21     }
22 }
23
24 void update(int pos, int val, int in = 1, int L = 0, int R = n - 1) {
25     if (L == R) {
26         ST[in].num = val;
27     } else {
28         int mid = (L + R) >> 1;
29         if (pos <= mid) {
30             update(pos, val, 2 * in, L, mid);
```

```

31         } else {
32             update(pos, val, (2 * in) + 1, mid + 1, R);
33         }
34         ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
35     }
36 }
37
38 info get(int l, int r, int in = 1, int L = 0, int R = n - 1) {
39     if (r < L or R < l) return {0}; // Elemento neutro
40     if (l <= L and R <= r) return ST[in];
41
42     int mid = (L + R) >> 1;
43     info left = get(l, r, 2 * in, L, mid);
44     info right = get(l, r, (2 * in) + 1, mid + 1, R);
45     return ope(left, right);
46 }
47
48 // Llamadas simples y limpias:
49 // build(v); // Construye con el vector
50 // update(2, 10); // Cambia el valor en pos 2 a 10
51 // info ans = get(1, 3); // Obtiene la respuesta en el rango [1, 3]
52 // v = {1 2 3 4 5}
53 // get(1, 0, n-1, 1, 3) = 2 + 3 + 4 = 9
54

```

5.2. Segment Tree Lazy Propagation

```

1  const int N = 1e5;
2
3  struct info {
4      long long num;
5  };
6
7  info ST[4 * N];
8  long long lazy[4 * N];
9
10 info ope(info u, info v) {
11     return {u.num + v.num};
12 }
13
14 void build(const vector<int> &v, int in = 1, int L = 0, int R = n - 1) {
15     if (L == R) {
16         ST[in] = {v[L]};
17     } else {
18         int mid = (L + R) >> 1;
19
20         build(v, 2 * in, L, mid);
21         build(v, 2 * in + 1, mid + 1, R);
22
23         ST[in] = ope(ST[2 * in], ST[2 * in + 1]);
24     }
25 }
26
27 void push(int in, int L, int R) {
28     if (lazy[in] == 0)
29         return;
30
31     ST[in].num += lazy[in] * (R - L + 1);
32
33     if (L != R) {
34         lazy[2 * in] += lazy[in];
35         lazy[2 * in + 1] += lazy[in];
36     }
37
38     lazy[in] = 0;
39 }
40
41 void update(int l, int r, long long val,
42             int in = 1, int L = 0, int R = n - 1) {
43
44     push(in, L, R);
45
46     if (r < L or R < l)
47         return;
48
49     if (l <= L and R <= r) {
50         lazy[in] += val;
51         push(in, L, R);
52         return;
53     }
54
55     int mid = (L + R) >> 1;
56
57     update(l, r, val, 2 * in, L, mid);
58

```

```

58     update(l, r, val, 2 * in + 1, mid + 1, R);
59     ST[in] = ope(ST[2 * in], ST[2 * in + 1]);
60 }
61
62 info get(int l, int r,
63         int in = 1, int L = 0, int R = n - 1) {
64     push(in, L, R);
65
66     if (r < L or R < l)
67         return {0};
68
69     if (l <= L and R <= r)
70         return ST[in];
71
72     int mid = (L + R) >> 1;
73
74     info left = get(l, r, 2 * in, L, mid);
75     info right = get(l, r, 2 * in + 1, mid + 1, R);
76
77     return ope(left, right);
78 }
79
80 }

```

5.3. BIT Fenwick Tree

```

1  struct BIT{
2      int n;
3      vector<int> bit;
4
5      BIT(int _n){
6          n = _n;
7          bit.assign(n+1,0);
8      }
9
10     void add(int i, int val){
11         while(i<=n){
12             bit[i] += val;
13             i += (i&(-i));
14         }
15     }
16
17     int sum(int i){
18         int ans = 0;
19         while(i > 0){
20             ans += bit[i];
21             i -= (i&(-i));
22         }
23         return ans;
24     }
25
26     int query(int l, int r){
27         return sum(r) - sum(l-1);
28     }
29 };
30 //se debe utilizar con index 1
31 //Sabrossus

```

5.4. Bit 2D

```

1  struct BIT_2D{
2      vector<vector<int>> bit;
3      int n,m;
4
5      BIT_2D(int x, int y){
6          bit.assign(x+1,vector<int> (y+1,0));
7          n = x;
8          m = y;
9      }
10
11     void update(int x, int y, int val){
12         for(int i = x ; i <= n ; i+=i&(-i)){
13             for(int j= y; j <=m ; j+=j&(-j)){
14                 bit[i][j] += val;
15             }
16         }
17     }
18
19     int sum(int x, int y){
20         int ans = 0;
21         for(int i = x ; i >0 ; i-=i&(-i)){

```

```

22         for(int j= y; j >0 ; j-=j&-j){
23             ans += bit[i][j];
24         }
25     }
26     return ans;
27 }
28
29 int query(int x1, int y1, int x2, int y2){
30     return sum(x2,y2) + sum(x1-1,y1-1) - sum(x1-1,y2) - sum(x2,y1-1);
31 }
32 };
33
34 void solve(){
35     int n,q;
36     cin >> n >> q;
37     BIT_2D bit(n,n);
38     vector<vector<int>> mt(n+1,vector<int>(n+1,0));
39     for(int i = 1 ; i <= n; i ++){
40         for(int j = 1 ; j <= n ;j++){
41             char uwu;
42             cin >> uwu;
43             if(uwu=='.' ) continue;
44             mt[i][j] = 1;
45             bit.update(i,j,1);
46         }
47     }
48     while(q--){
49         int tipo;
50         cin >> tipo;
51         if(tipo == 1){
52             int x,y;
53             cin >> x >> y;
54             if(mt[x][y]){
55                 bit.update(x,y,-1);
56             }
57             else{
58                 bit.update(x,y,1);
59             }
60             mt[x][y] ^= 1;
61         }
62         else{
63             int x1,y1,x2,y2;
64             cin >> x1 >> y1 >> x2 >> y2;
65             cout << bit.query(x1,y1,x2,y2) << '\n';
66         }
67     }
68 }
69 //No entiendo w, el dildan lo hizo

```

5.5. Disjoint Set Union

```

1  struct DSU{
2      int num;
3      vector<int> parent;
4      vector<int> sizes;
5      DSU(){};
6      DSU(int n){
7          init(n);
8      }
9      void init(int n){
10         num = n;
11         parent.resize(n+1);
12         sizes.resize(n+1);
13         //si tienes index 0 inicializar desde 0
14         for(int i = 1; i <= n ; i++){
15             parent[i] = i;
16             sizes[i] = 1;
17         }
18     }
19     int find(int a){
20         if(a == parent[a]) return a;
21         return parent[a] = find(parent[a]);
22     }
23     void join(int a, int b){
24         int x = find(a);
25         int y = find(b);
26         if(x == y) return;
27         num--;
28         if(sizes[x] < sizes[y]){
29             parent[x] = y;
30             sizes[y] += sizes[x];
31         }
32         else{
33             parent[y] = x;

```

```

34         sizes[x] += sizes[y];
35     }
36 }
37 };
38 //Sabroscus
39

```

5.6. Implicit Treap

```

1  // Mantiene las posiciones
2  struct Node {
3      int val , pri , sz ;
4      Node *l , *r;
5      Node ( int v ) {
6          val = v;
7          pri = rand () ;
8          sz = 1;
9          l = r = nullptr ;
10     }
11 };
12 using pnode = Node *;
13 int sz ( pnode t ) {
14     return t ? t -> sz : 0;
15 }
16 void upd (pnode t) {
17     if (t) t -> sz = 1 + sz (t -> l) + sz (t -> r);
18 }
19 void split ( pnode t , pnode &l , pnode &r , int k ) {
20     if (!t) {
21         l = r = nullptr ;
22         return ;
23     }
24     int leftSize = sz (t -> l);
25     if ( leftSize >= k ) {
26         split (t -> l , l , t -> l , k) ;
27         r = t;
28     }
29     else {
30         split (t -> r , t -> r , r , k - leftSize - 1) ;
31         l = t;
32     }
33     upd (t);
34 }
35 pnode merge ( pnode l , pnode r ) {
36     if (!l || !r ) return l ? l : r;
37
38     if (l -> pri > r -> pri ) {
39         l -> r = merge (l -> r , r);
40         upd (l );
41         return l;
42     }
43     else {
44         r -> l = merge (l , r -> l );
45         upd (r );
46         return r;
47     }
48 }
49 void insert ( pnode &root , int pos , int val ) {
50     pnode a , b;
51     split ( root , a , b , pos );
52     root = merge ( merge (a , new Node ( val )) , b) ;
53 }
54 void erase ( pnode &root , int pos ) {
55     pnode a , b , c;
56     split ( root , a , b , pos );
57     split (b , b , c , 1);
58     root = merge (a , c) ;
59 }
60 void print(pnode t) {
61     if (!t) return;
62
63     print(t->l);
64     cout << t->val << "□";
65     print(t->r);
66 }
67 int get(pnode t, int k) {
68     if (!t) return -1;
69
70     int leftSize = sz(t->l);
71
72     if (k < leftSize)
73         return get(t->l, k);
74
75     if (k == leftSize)

```

```

76     return t->val;
77
78     return get(t->r, k - leftSize - 1);
79 }

```

5.7. Ordered Set

```

1  #include <bits/stdc++.h>
2  #include <ext/pb_ds/assoc_container.hpp>
3  #include <ext/pb_ds/tree_policy.hpp>
4
5  using namespace std;
6  using namespace __gnu_pbds;
7
8  typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>
9      ordered_set;
10
11 void solve() {
12     ordered_set st;
13
14     st.insert(10);
15     st.insert(30);
16     st.insert(20);
17
18     // 1. Elemento en la posición K (0-indexed, de menor a mayor)
19     // find_by_order devuelve un iterador (puntero). Usamos '*' para el valor.
20     auto it = st.find_by_order(1);
21     if (it != st.end()) {
22         int elemento = *it; // Devuelve 20 de forma segura
23     }
24
25     // 2. Cuantos elementos son estrictamente menores que X
26     int menores_que_25 = st.order_of_key(25); // Devuelve 2 (el 10 y el 20)
27 }
// Sabrossus

```

5.8. Sparse Table

```

1  const int N = 1e5;
2  const int LOG = 20;
3  int v[N]; //values
4  int st[N][LOG]; //Sparse Table
5  int lg[N]; // log values
6  int n;
7  void build(){
8      for(int i = 0; i < n; i++) st[i][0] = v[i];
9
10     for(int i = 1; i < LOG; i++){
11         for(int j = 0; j < n - (1<<i) + 1; j++){
12             st[j][i] = min(st[j][i-1], st[j+(1<<(i-1))][i-1]);
13         }
14     }
15     lg[1] = 0;
16     for(int i = 2; i <= n; i++) lg[i] = lg[i/2] + 1;
17 }
18 int query(int L, int R){
19     int k = lg[R-L+1];
20     return min(st[L][k], st[R-(1<<k)+1][k]);
21 }
22 // v = [5, 2, 4, 7, 1, 3]
23 // query(1, 4)      [2, 4, 7, 1]

```

5.9. Treap

```

1  // Ordena los elementos por valor
2  struct Node {
3      int key, pri, sz;
4      Node *l, *r;
5      Node(int k) {
6          key = k;
7          pri = rand();
8          sz = 1;
9          l = r = nullptr;
10     }
11 };
12
13 using pnode = Node*;

```

```
14
15 int sz(pnode t) {
16     return t ? t->sz : 0;
17 }
18
19 void upd(pnode t) {
20     if (t) t->sz = 1 + sz(t->l) + sz(t->r);
21 }
22
23 void split(pnode t, pnode &l, pnode &r, int k) {
24     if (!t) {
25         l = r = nullptr;
26         return;
27     }
28
29     int leftSize = sz(t->l);
30
31     if (leftSize >= k) {
32         split(t->l, l, t->l, k);
33         r = t;
34     } else {
35         split(t->r, t->r, r, k - leftSize - 1);
36         l = t;
37     }
38
39     upd(t);
40 }
41
42 pnode merge(pnode l, pnode r) {
43     if (!l || !r) return l ? l : r;
44
45     if (l->pri > r->pri) {
46         l->r = merge(l->r, r);
47         upd(l);
48         return l;
49     } else {
50         r->l = merge(l, r->l);
51         upd(r);
52         return r;
53     }
54 }
55
56 void insert(pnode &root, int pos, int val) {
57     pnode a, b;
58     split(root, a, b, pos);
59     root = merge(merge(a, new Node(val)), b);
60 }
61
62 void erase(pnode &root, int pos) {
63     pnode a, b, c;
64     split(root, a, b, pos);
65     split(b, b, c, 1);
66     root = merge(a, c);
67 }
68
69 int get(pnode t, int k) {
70     if (!t) return -1;
71
72     int leftSize = sz(t->l);
73
74     if (k < leftSize)
75         return get(t->l, k);
76
77     if (k == leftSize)
78         return t->key;
79
80     return get(t->r, k - leftSize - 1);
81 }
82
83 void print(pnode t) {
84     if (!t) return;
85
86     print(t->l);
87     cout << t->key << " ";
88     print(t->r);
89 }
```

6. Dynamic Programing

6.1. Bounded Knapsack

```

1 // bounded_knapsack para DPs del estilo :
2 // dp[j - cw] + cv , w-> pesos-> valor y quieres maximizar el valor
3 int n, we;
4 cin >> n >> we;
5 vector<int> v, m, w, dp;
6 v.resize(n);
7 m.resize(n);
8 w.resize(n);
9 dp.assign(we+1, 0);
10 int sum = 0;
11 for(int i = 0 ; i < n ; i++){
12     int a, b, c;
13     cin >> a >> b >> c;
14     v[i] = a;
15     w[i] = b;
16     m[i] = c;
17     vector<int> old = dp;
18     for(int r = 0; r < w[i] and r <= we ; r++){
19         deque<pair<int,int>> dq;
20         for(int j = r; j <= we ; j += w[i]){
21             int k = (j - r)/w[i];
22             while(!dq.empty() and dq.front().first < k - m[i]){
23                 dq.pop_front();
24             }
25             int cur = old[j] - k * v[i];
26             while(!dq.empty() && dq.back().second <= cur){
27                 dq.pop_back();
28             }
29             dq.push_back({k, cur});
30             dp[j] = dq.front().second + k * v[i];
31         }
32     }
33 }
34 cout << dp[we];

```

6.2. SOS DP

```

1 void sos_dp(vector<ll>& dp, int K){
2     // k es el tamaño del arreglo :v
3     for(int bit = 0; bit < K; bit++){
4         for(int mask = 0; mask < (1 << K); mask++){
5             if(mask & (1 << bit)){
6                 dp[mask] += dp[mask ^ (1 << bit)];
7                 // si quieres que sea un sos dp de maximos
8                 // dp[mask] = max(dp[mask], dp[mask ^ (1 << bit)]);
9             }
10        }
11    }
12 }

```

6.3. Fastfib

```

1 #include <bits/stdc++.h>
2 #define ll long long
3
4 using namespace std;
5
6 const ll MOD = 1e9 + 7;
7
8 struct Matrix {
9     ll a[2][2];
10    Matrix() {
11        memset(a, 0, sizeof(a));
12    }
13 };
14
15 Matrix multiply(Matrix A, Matrix B) {
16     Matrix C;
17     for (int i = 0; i < 2; i++) {
18         for (int j = 0; j < 2; j++) {
19             for (int k = 0; k < 2; k++) {
20                 C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
21             }
22         }
23     }
24     return C;
25 }
26
27 Matrix power(Matrix A, ll n) {
28     Matrix R;
29     R.a[0][0] = 1;

```

```

30     R.a[1][1] = 1;
31     while (n) {
32         if (n & 1)
33             R = multiply(R, A);
34         A = multiply(A, A);
35         n >>= 1;
36     }
37     return R;
38 }
39
40 ll fibonacci(ll n) {
41     if (n == 0) return 0;
42     Matrix A;
43     A.a[0][0] = 1;
44     A.a[0][1] = 1;
45     A.a[1][0] = 1;
46     A.a[1][1] = 0;
47     Matrix R = power(A, n - 1);
48     return R.a[0][0];
49 }
50
51 int main() {
52     ll n;
53     cin >> n;
54     cout << fibonacci(n) << '\n';
55 }

```

7. Flow

7.1. Dinic

```

1  struct Dinic
2  {
3      const int INF = 1e9;
4
5      struct flowEdge
6      {
7          int to, rev, f, cap;
8      };
9
10     vector<vector<flowEdge>> G;
11     vector<int> work, lvl, Q;
12
13     int S, T, nodes;
14
15     Dinic(){}
16     Dinic(int n, int s, int t){
17         S = s;
18         T = t;
19         nodes = n;
20         G.clear();
21         G.resize(n);
22         Q.resize(n);
23     }
24     void addEdge(int st, int en, int cap){
25         flowEdge A = {en, (int)G[en].size(), 0, cap};
26         flowEdge B = {st, (int)G[st].size(), 0, 0};
27         G[st].pb(A);
28         G[en].pb(B);
29     }
30     int dfs(int v, int f){
31         if(v == T or f == 0) return f;
32         for(int &i = work[v]; i < G[v].size(); i++){
33             flowEdge &e = G[v][i];
34             int u = e.to;
35             if(e.cap <= e.f or lvl[u] != lvl[v] + 1) continue;
36             int df = dfs(u, min(f, e.cap - e.f));
37             if(df){
38                 e.f += df;
39                 G[u][e.rev].f -= df;
40                 return df;
41             }
42         }
43         return 0;
44     }
45     bool bfs(){
46         int qt = 0;
47         Q[qt++] = S;
48         lvl.assign(nodes, -1);
49         lvl[S] = 0;
50         for(int i = 0; i < qt; i++){

```

```

51         int u = Q[i];
52         for(flowEdge &e : G[u]){
53             int v = e.to;
54             if(e.cap <= e.f or lvl[v] != -1) continue;
55             lvl[v] = lvl[u] + 1;
56             Q[qt++] = v;
57         }
58     }
59     return lvl[T] != -1;
60 }
61 int maxFlow(){
62     int flow = 0;
63     while(bfs()){
64         work.assign(nodes,0);
65         while(true){
66             int df = dfs(S,INF);
67             if(df == 0) break;
68             flow += df;
69         }
70     }
71     return flow;
72 }
73
74 };

```

8. Graph Algorithms

8.1. Dijkstra

```

1  const int N = 1e5;
2
3  int vis[N];
4  vector<pair<int,int>> G[N];
5
6  void dijkstra(int x){
7      priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
8      pq.push({0,x});
9      vis[x] = 0;
10     while(!pq.empty()){
11         auto y = pq.top();
12         pq.pop();
13         int u = y.second;
14         int cost = y.first;
15         for(auto v : G[u]){
16             int cur = cost + v.second;
17             if(cur < vis[v.first]){
18                 vis[v.first] = cur;
19                 pq.push({cur,v.first});
20             }
21         }
22     }
23 }

```

8.2. Kruskal MST

```

1  const int N = 1e5;
2  struct DSU{
3      int num;
4      vector<int> parent;
5      vector<int> sizes;
6      DSU(){};
7      DSU(int n){
8          init(n);
9      }
10     void init(int n){
11         num = n;
12         parent.resize(n+1);
13         sizes.resize(n+1);
14         for(int i = 1; i <= n ; i++){
15             parent[i] = i;
16             sizes[i] = 1;
17         }
18     }
19     int find(int a){
20         if(a == parent[a]) return a;
21         return parent[a] = find(parent[a]);
22     }
23     void join(int a, int b){

```

```
24     int x = find(a);
25     int y = find(b);
26     if(x == y) return;
27     num--;
28     if(sizes[x] < sizes[y]){
29         parent[x] = y;
30         sizes[y] += sizes[x];
31     }
32     else{
33         parent[y] = x;
34         sizes[x] += sizes[y];
35     }
36 }
37 };
38 int n;
39 vector<pair<int, pair<int, int>>> List;
40 int Kruskal_MST(){
41     sort(List.begin(), List.end());
42     DSU dsu(n);
43     int mst = 0, t = 1;
44     for(int i = 0; i < List.size(); i++){
45         int w = List[i].first; //weight
46         int a = List[i].second.first; //at
47         int b = List[i].second.second; //to
48         if(dsu.find(a) != dsu.find(b)){
49             dsu.join(a, b);
50             mst += w;
51             t++;
52         }
53         if(t == n) break;
54     }
55     return mst;
56 }
```

8.3. Topological Sort Kahn Algorithm

```
1  const int N = 1e5;
2
3  int indegree[N];
4  vector<int> G[N];
5  int n;
6  vector<int> Kahn(){
7      queue<int> q;
8      vector<int> ans;
9      for(int i = 0; i < n; i++){
10         if(indegree[i] == 0) q.push(i);
11     }
12     while (!q.empty()){
13         int v = q.front();
14         q.pop();
15         ans.push_back(v);
16         for(int u : G[v]){
17             indegree[u]--;
18             if(indegree[u] == 0){
19                 q.push(u);
20             }
21         }
22     }
23     //if(ans.size() != n) grafo no DAG
24     return ans;
25 }
```

9. Tree Algorithms

9.1. Euler Tour

```
1  const int N = 1e5 + 10;
2  vector<int> path;
3  vector<int> G[N];
4  void EulerTour(int node, int parent){
5      path.pb(node);
6      for(int u : G[node]){
7         if(u != parent){
8             EulerTour(u, node);
9         }
10     }
11     path.pb(node);
12 }
```

```

13
14 const int N = 1e5 + 10;
15 int tin[N];
16 int tout[N];
17 int flat[N];
18 int timer = 1;
19 void EulerTour(int node, int parent){
20     tin[node] = timer;
21     flat[timer] = node;
22     timer++;
23     for(int u : G[node]){
24         if(u != parent){
25             EulerTour(u, node);
26         }
27     }
28     tout[node] = timer;
29 }

```

9.2. Binary Lifting

```

1  const int N = 1e5;
2  const int LOG = 20;
3  int up[N][LOG];
4  vector<int> G[N];
5  //Find parents
6  void dfs(int u){
7      for(int v : G[u]){
8          up[v][0] = u;
9          for(int j = 1; j < LOG; j++){
10             up[v][j] = up[up[v][j-1]][j-1];
11         }
12         dfs(v);
13     }
14 }
15 int get_K_parent(int a, int k){
16     for(int j = LOG-1; j >= 0; j++){
17         if(k & (1<<j)){
18             a = up[a][j];
19         }
20     }
21     return a;
22 }

```

9.3. Centroid Decomposition

```

1  //EJERCICIO IMPLEMENTACION XENIA AND TREE
2  //Nodo rojo mas cercano a un nodo v
3
4  const int N = 2e5+10;
5  vector<int> G[N];
6  int tag[N]; // time of discovery of centroid
7  int fat[N]; // father in centroid decomposition
8  int szt[N]; // size of subtree
9
10 int best[N];
11 int niv[N];
12 int dist[30][N]; //dist[nivel][nodo];
13
14 void dfs_level(int node, int p, int niv, int prof){
15     dist[niv][node] = prof;
16     for(auto y : G[node]){
17         if(y != p and tag[y] < 0){
18             dfs_level(y, node, niv, prof+1);
19         }
20     }
21 }
22 // -----CENTROID-----
23 int calcsz(int node, int p){
24     szt[node] = 1;
25     for(auto y : G[node]){
26         if(y != p and tag[y] < 0){
27             szt[node] += calcsz(y, node);
28         }
29     }
30     return szt[node];
31 }
32 int ccnt = 0;
33 void cdfs(int node, int p, int nivel = 0, int sz = -1){
34     if(sz < 0) sz = calcsz(node, -1);
35     for(auto y : G[node]){
36         if(tag[y] < 0 and szt[y] * 2 >= sz){

```

```

37         szt[node] = 0;
38         cdfs(y, p, nivel, sz);
39         return;
40     }
41 }
42 tag[node] = ccnt++;
43 fat[node] = p;
44 niv[node] = nivel;
45 dfs_level(node, -1, nivel, 0);
46 for(auto y : G[node]){
47     if(tag[y] < 0){
48         cdfs(y, node, nivel+1);
49     }
50 }
51 }
52
53 int n;
54 void centroid(){
55     ccnt = 0;
56     repl(i,1,n+1){
57         tag[i] = -1;
58     }
59     cdfs(1, -1);
60 }
61
62 //-----
63
64 void update(int node, int rojo){
65     int nivel = niv[node];
66     int cost = dist[nivel][rojo];
67     best[node] = min(best[node], cost);
68     int p = fat[node];
69     if(p > 0){
70         update(p, rojo);
71     }
72 }
73
74 int improve(int node, int cur){
75     if(node <= 0){
76         return 1e9;
77     }
78     int nivel = niv[node];
79     int cost = best[node] + dist[nivel][cur];
80     int p = fat[node];
81     return min(cost, improve(p, cur));
82 }
83
84 void solve(){
85     int qq;
86     cin >> n >> qq;
87     repl(i,0,30){
88         repl(j,1,n+1){
89             dist[i][j] = -1;
90             best[j] = 1e9;
91         }
92     }
93     repl(i,0,n-1){
94         int a, b;
95         cin >> a >> b;
96         G[a].push_back(b);
97         G[b].push_back(a);
98     }
99     centroid();
100    update(1, 1);
101    while(qq--){
102        int q, node;
103        cin >> q >> node;
104        if(q == 1){
105            update(node, node);
106        }
107        else{
108            int res = best[node];
109            int res2 = improve(node, node);
110            cout << min(res, res2) << '\n';
111        }
112    }
113 }
114
115 int main(){
116     velocito;
117     int t = 1;
118     // cin >> t;
119     while(t--){
120         solve();
121     }
122     return 0;

```

```
123 }
124 //Sabrossus
125 //TC Argentina
```

9.4. Lowest Common Ancestor

```
1  const int N = 1e5;
2  const int LOG = 20;
3  vector<int> G[N];
4  int depth[N];
5  int up[N][LOG];
6  // Assign levels and get parents
7  void dfs(int u){
8      for(int v : G[u]){
9          depth[v] = depth[u] + 1;
10         up[v][0] = u;
11         for(int j = 1; j < LOG; j++){
12             up[v][j] = up[up[v][j-1]][j-1];
13         }
14         dfs(v);
15     }
16 }
17 int get_lca(int a, int b){
18     if(depth[a] < depth[b]) swap(a,b);
19     int k = depth[a] - depth[b];
20     for(int j = LOG-1; j >= 0; j--){
21         if(k & (1<<j)){
22             a = up[a][j];
23         }
24     }
25     if(a == b) return a;
26     for(int j = LOG-1; j >= 0; j--){
27         if(up[a][j] != up[b][j]){
28             a = up[a][j];
29             b = up[b][j];
30         }
31     }
32     return up[a][0];
33 }
```

10. Utilities ---

10.1. Int128

```
1  // --- LECTURA B SICA ---
2  __int128 read128() {
3      string s;
4      cin >> s;
5      __int128 n = 0;
6      int sign = 1;
7      size_t start = 0;
8
9      if (!s.empty() && s[0] == '-') {
10         sign = -1;
11         start = 1;
12     }
13
14     for (size_t i = start; i < s.length(); ++i) {
15         n = n * 10 + (s[i] - '0');
16     }
17
18     return n * sign;
19 }
20
21 // --- IMPRESI N B SICA ---
22 void print128(__int128 n) {
23     if (n == 0) {
24         cout << 0;
25         return;
26     }
27     if (n < 0) {
28         cout << '-';
29         n = -n;
30     }
31     string s;
32     while (n > 0) {
33         s += (char)('0' + (n % 10));
34         n /= 10;
```

```

35     }
36     reverse(s.begin(), s.end());
37     cout << s;
38 }

```

10.2. Merge Sort

```

1  // Rango de uso: 0-indexed.
2  // Llamar en el solve como: mergeSort(v, 0, n - 1);
3  ll ans = 0;
4
5  void mergeSort(vector<ll> &v, int low, int high) {
6      if (low == high) return;
7
8      int mid = (low + high) >> 1;
9      mergeSort(v, low, mid);
10     mergeSort(v, mid + 1, high);
11
12     queue<ll> L, H;
13     for (int i = low; i < mid + 1; i++) {
14         L.push(v[i]);
15     }
16     for (int i = mid + 1; i < high + 1; i++) {
17         H.push(v[i]);
18     }
19
20     for (int i = low; i < high + 1; i++) {
21         if (L.size() == 0) {
22             v[i] = H.front();
23             H.pop();
24         }
25         else if (H.size() == 0) {
26             v[i] = L.front();
27             L.pop();
28         }
29         else {
30             if (L.front() <= H.front()) {
31                 v[i] = L.front();
32                 L.pop();
33             }
34             else {
35                 v[i] = H.front();
36                 H.pop();
37                 ans += L.size(); // Inversion detectada
38             }
39         }
40     }
41 }

```

10.3. Prefix Sum 2D

```

1  //1 indexed
2  const int N = 1005;
3  int v[N][N];
4  int acu[N][N];
5  int n, m;
6
7  void solve() {
8      //limpiar la acu para cada TC
9
10     // Construcción: acumula 1 si el elemento es exactamente 1
11     for (int i = 1; i <= n; i++) {
12         for (int j = 1; j <= m; j++) {
13             int val = (v[i][j] == 1 ? 1 : 0);
14             acu[i][j] = val + acu[i - 1][j] + acu[i][j - 1] - acu[i - 1][j - 1];
15         }
16     }
17
18     // Esquinas de consulta:
19     // (a, b) -> Superior Izquierda
20     // (c, d) -> Inferior Derecha
21     int a = 2, b = 2, c = 4, d = 4; // Valores de ejemplo
22
23     int ans = acu[c][d] - acu[a - 1][d] - acu[c][b - 1] + acu[a - 1][b - 1];
24     cout << ans << '\n';
25 }

```

10.4. Random

```

1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2 // mt19937_64 para 64 bits usar uint64_t o unsigned long long
3 cout << rng() << endl;
4 shuffle(v.begin(),v.end(),rng);

```

10.5. Plantilla Python

```

1 import sys
2 from bisect import bisect_left, bisect_right, insort
3 from collections import defaultdict, Counter, deque
4 from heapq import heappush, heappop, heapify
5 from math import gcd, lcm, isqrt
6
7 input = sys.stdin.readline
8
9
10 def solve():
11     n = int(input()) #leer numero
12     a,b,c= map(int, input().split()) #leer numeros
13     s = input().strip() #leer cadenas
14     v = list(map(int, input().split())) #leer lista de enteros
15
16
17 t = 1
18 # t = int(input())
19 for _ in range(t):
20     solve()
21
22 # C++                                Python
23
24 # vector                             list
25 # array                             list
26 # string                             str
27
28 # set                               set
29 # map                               dict
30 # multiset                           Counter
31 # unordered_map                       dict
32 # unordered_set                       set
33
34 # priority_queue                     heapq
35 # queue                             deque
36 # deque                             deque
37
38 # lower_bound                         bisect_left
39 # upper_bound                         bisect_right
40 # sort                               list.sort()
41 # binary_search                       bisect_left/right
42
43 # pair                               tuple
44 # struct                             class / dataclass
45
46 # gcd                               math.gcd
47 # lcm                               math.lcm
48
49 # push_back                           append
50 # push_front                         appendleft
51 # pop_back                           pop
52 # pop_front                         popleft
53
54 # size()                             len()
55 # empty()                           not x

```

10.6. Pragma

```

1 #pragma GCC optimize("O3,unroll-loops")
2 #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")

```